

Interview Questions — Java - Software Design Pattern And Project

Total Questions: 46

1. What is a software design pattern, and why are design patterns important in Java web development?

Hint: Think about code maintainability and avoiding repeated solutions to common problems.

2. Explain the difference between creational, structural, and behavioral design patterns.

Hint: Consider what aspect of object-oriented design each category addresses.

3. What is the Singleton pattern, and why would you use it for a database connection pool in a Java application?

Hint: Focus on controlling the number of instances and resource management.

4. How does the Factory pattern help in object creation? Give a real-world example relevant to Java web apps.

Hint: Think about situations where you want to hide the instantiation logic.

5. What is the DAO (Data Access Object) pattern, and how does it help separate concerns in a Java web application?

Hint: Consider how data access logic is isolated from business logic.

6. Describe the main components of the MVC (Model-View-Controller) pattern as applied in Java Servlets and JSP.

Hint: Identify which part handles user input, business logic, and output.

7. In a simple user management app, how would the UserDao class interact with the controller servlet?

Hint: Think about method calls for user data retrieval or update.

8. Walk through the flow of a login operation using the MVC + DAO pattern in a Java web app.

Hint: Start from form submission and end with the user being logged in or shown an error.

9. What are the main advantages of using abstraction and reusability in the DAO layer?

Hint: Consider maintainability and flexibility for future changes.

10. How does the MVC + DAO pattern provide a foundation for frameworks like Spring MVC?

Hint: Think about how responsibilities are separated and how this maps to Spring.

11. Why is session tracking needed in Java web applications?

Hint: Focus on the stateless nature of HTTP and user-specific data.

12. Explain how the HttpSession API works for session management in servlets.

Hint: Mention session creation, attribute storage, and session invalidation.

13. Compare session tracking using cookies and hidden form fields.

Hint: Think about where data is stored and visibility to the client.

14. What are some best practices for session security in Java web applications?

Hint: Consider session hijacking, timeouts, and secure cookie flags.

15. How does session.invalidate() differ from JWT-based token expiry?

Hint: Compare server-side and stateless client-side session management.

16. Describe how you would integrate a Bootstrap HTML template with a Java Servlet-based web application.

Hint: Focus on folder structure and linking static resources.

17. How do you separate static resources like CSS, JS, and images from dynamic content in a Java web app?

Hint: Think about the recommended folder structure.

18. What is the role of JSP, JSTL, and EL in rendering dynamic content in Java web applications?

Hint: Consider how data is passed from the backend to the view.

19. If you are building a professional-grade web app, what folder structure would you follow for your templates and static files?

Hint: Think about organizing files for maintainability and scalability.

20. How would you handle file uploads in a Java Servlet using MultipartConfig?

Hint: Mention annotation/configuration and processing uploaded files.

21. What are the pros and cons of saving uploaded files as blobs in the database versus saving file paths?

Hint: Consider storage efficiency, backup, and retrieval performance.

22. Write a code snippet to handle image upload and save the file to a specific folder on the server using Java Servlets.

Hint: Use `MultipartConfig` and `getPart()`.

23. How would you implement a download servlet to allow users to download files they previously uploaded?

Hint: Focus on setting response headers and streaming file content.

24. Explain the basic concept of SMTP and how it is used with the JavaMail API.

Hint: Think about sending emails from a web application.

25. Describe how you would send an HTML email with an attachment using JavaMail.

Hint: Mention message composition, multipart content, and transport.

26. What is the difference between transactional and marketing emails?

Hint: Consider the purpose and content of each type.

27. How would you generate a random OTP in Java for an email verification system?

Hint: Think about using the `Random` class or `SecureRandom`.

28. Where would you store an OTP temporarily for verification—session or database? Why?

Hint: Consider data lifetime and security.

29. Describe how you would implement OTP expiry and what best practices you would follow.

Hint: Focus on time limits and handling expired OTPs.

30. How can you use ChatGPT or LangChain4J to auto-generate an OTP logic function, and what would you check before using the code in production?

Hint: Think about code review, testing, and security.

31. What is a payment gateway, and why do we need to integrate one (like Razorpay or Stripe) in a Java web application?

Hint: Focus on handling online transactions securely.

32. Describe the flow of integrating a payment button (e.g., Razorpay) with a Spring Boot backend.

Hint: Mention front-end, API calls, and backend verification.

33. How do you securely manage API keys when integrating payment gateways?

Hint: Consider environment variables, config files, and not exposing keys in code.

34. What is webhook verification, and why is it important in payment gateway integrations?

Hint: Focus on confirming payment status and preventing fraud.

35. How could you use ChatGPT to help you test payment API response payloads, and what should you verify before trusting the AI's output?

Hint: Think about data accuracy and matching actual gateway documentation.

36. What is AJAX, and how does it improve the user experience in web applications?

Hint: Consider page reloads and asynchronous data updates.

37. How would you use AJAX to check if an email already exists before allowing user registration in a Java web app?

Hint: Mention client-side request and server-side validation.

38. Write a simple JavaScript fetch() call that submits a form asynchronously to a servlet endpoint.

Hint: Use fetch with method POST and handle the response.

39. How does the concept of AJAX relate to using Axios in a React-based project?

Hint: Think about asynchronous HTTP requests in both cases.

40. What is CI/CD, and why is it important for modern Java application development?

Hint: Focus on automation, faster releases, and reliability.

41. Describe the typical steps in a CI/CD pipeline for a Java web application.

Hint: Mention build, test, and deploy stages.

42. How would you set up a simple GitHub Actions workflow to build and deploy a Spring Boot app?

Hint: Think about YAML configuration and deployment steps.

43. What is the role of automated testing (e.g., JUnit) in a CI/CD pipeline?

Hint: Consider catching bugs before deployment.

44. Integrative: How would you combine the DAO pattern, session management, and AJAX to create a secure and responsive login system in a Java web application?

Hint: Think about backend separation, user authentication, and real-time feedback.

45. Integrative: If you are asked to build a registration system that sends an OTP email and supports file upload for profile pictures, which patterns and technologies from this module would you use, and how would you organize the flow?

Hint: Consider DAO, JavaMail, session, and file upload handling.

46. Integrative: How can you ensure that integrating external APIs (like payment gateways) does not break your deployment pipeline, and what tools from this module would help you automate testing and deployment?

Hint: Think about CI/CD, automated tests, and API mocking.